

Web Programming (CS4032)

Course Instructor(s):

Dr. Tajwar Mehmood

Section(s): A,B,C

Sessional-II Exam

Total Time (Hrs): 1

Total Marks: 20

Total Questions: 3

Date: Apr 9, 2026

Roll No

Course Section

Student Signature

Do not write below this line.

Attempt all the questions.

CLO 1. Explain the role and significance of modern web application development technologies, tools, and industry best practices.

CLO 2. Design and develop dynamic, responsive, and scalable web applications using modern front-end and back-end frameworks.

NOTE: Complete the missing parts (____) to create a simple asked task.

[CLO 1: Demonstrate the basic concepts of programming]

Q1: [10 marks]

Answer:

Parent Component: `parent.component.ts`

```
import { Component } from '@angular/core';
```

```
@Component({
  selector: 'app-parent',
  template: `
    <h2>Parent Component</h2>
    <app-child [childData]="parentMessage"></app-child>
  `,
})
export class ParentComponent {
  parentMessage = 'Hello from Parent!';
}
```

National University of Computer and Emerging Sciences

Islamabad Campus

Child Component: `child.component.ts`

```
import { Component, Input } from '@angular/core';
```

```
@Component({
  selector: 'app-child',
  template: `
    <h3>Child Component</h3>
    <p>Data from parent: {{ childData }}</p>
  `,
})
export class ChildComponent {
  @Input() childData!: string; // Receives data from parent
}
```

[CLO 2]

Q2: REST API using **Express.js** and **MongoDB (Mongoose)**

[20 marks]

Create a simple REST API using **Express.js** and **MongoDB (Mongoose)** in a single file.

Your application should:

1. Connect to a MongoDB database named **mydatabase**.
2. Define a schema called **User** with the following fields:
 - o `name` (String)
 - o `email` (String)
3. Create a POST API endpoint `/add-user` that:
 - o Accepts JSON data (`name` and `email`) from the request body
 - o Inserts the data into the MongoDB database
 - o Returns a success message along with the saved user data
4. Handle errors properly and return appropriate responses.
5. Run the server on port **3000**.

Fill in the following code

```
const express = require("express");
const mongoose = require("mongoose");
```

National University of Computer and Emerging Sciences

Islamabad Campus

```
const app = express();
app.use(express.json());

// MongoDB Connection
mongoose.connect("mongodb://127.0.0.1:27017/mydatabase", {
  useNewUrlParser: true,
  useUnifiedTopology: true,
})
.then(() => console.log("MongoDB Connected"))
.catch(err => console.log(err));

// Schema & Model
const UserSchema = new mongoose.Schema({
  name: String,
  email: String,
});

const User = mongoose.model("User", UserSchema);

// Insert Route
app.post("/add-user", async (req, res) => {
  try {
    const { name, email } = req.body;

    const newUser = new User({ name, email });
    const savedUser = await newUser.save();

    res.status(201).json({
      message: "User added successfully",
      data: savedUser,
    });
  }
});
```

National University of Computer and Emerging Sciences

Islamabad Campus

```
} catch (error) {  
  res.status(500).json({ error: error.message });  
}  
});
```

```
// 🚀 Start Server
```

```
app.listen(3000, () => {  
  console.log("Server running on port 3000");  
});
```

[CLO 3]

Q3: React application to manage product orders

[10 marks]

Create a simple React application to manage product orders. The app should demonstrate the use of components, props, state, and parent-child communication.

Requirements:

1. Components

- Create at least three components:
 - `ProductList` – displays a list of products.
 - `ProductItem` – represents a single product and has an "Add" button.
 - `OrderSummary` – displays the list of ordered items and the total price.

2. Parent-Child Communication

- Pass the list of products from the parent (`App`) to the child (`ProductList`) using props.
- Use a function passed as a prop to allow the child (`ProductItem`) to update the parent's state when a product is added to the order.

3. State Management

- Maintain the current order in the parent component (`App`) using `useState`.
- Update the order whenever a product is added.

4. Order Summary

- Display all ordered products and calculate the total price dynamically.

5. Styling

- Use simple CSS or inline styles to separate product list and order summary visually.

1. App Structure

```
src/  
├── App.js  
├── components/  
│   └── ProductList.js
```

```
| | ProductItem.js
| | OrderSummary.js
| | index.js
```

2. App.js

```
import React, { useState } from 'react';
import ProductList from './components/ProductList';
import OrderSummary from './components/OrderSummary';

const App = () => {
  const products = [
    { id: 1, name: 'Laptop', price: 800 },
    { id: 2, name: 'Phone', price: 500 },
    { id: 3, name: 'Headphones', price: 100 }
  ];

  const [order, setOrder] = useState([]);

  const addToOrder = (product) => {
    setOrder((prevOrder) => [...prevOrder, product]);
  };

  return (
    <div style={{ padding: '20px' }}>
      <h1>Order App</h1>
      <ProductList products={products} addToOrder={addToOrder} />
      <OrderSummary order={order} />
    </div>
  );
};

export default App;
```

3. ProductList.js

```
import React from 'react';
import ProductItem from './ProductItem';

const ProductList = ({ products, addToOrder }) => {
  return (
    <div>
      <h2>Products</h2>
      {products.map((product) => (
        <ProductItem key={product.id} product={product} addToOrder={addToOrder} />
      ))}
    </div>
  );
};
```

```
};  
  
export default ProductList;
```

4. ProductItem.js

```
import React from 'react';  
  
const ProductItem = ({ product, addToOrder }) => {  
  return (  
    <div style={{ marginBottom: '10px' }}>  
      <span>{product.name} - ${product.price}</span>  
      <button onClick={() => addToOrder(product)} style={{ marginLeft: '10px' }}>Add</button>  
    </div>  
  );  
};  
  
export default ProductItem;
```

5. OrderSummary.js

```
import React from 'react';  
  
const OrderSummary = ({ order }) => {  
  const total = order.reduce((sum, item) => sum + item.price, 0);  
  
  return (  
    <div style={{ marginTop: '20px' }}>  
      <h2>Order Summary</h2>  
      {order.length === 0 ? (  
        <p>No items in order</p>  
      ) : (  
        <ul>  
          {order.map((item, index) => (  
            <li key={index}>{item.name} - ${item.price}</li>  
          ))}  
        </ul>  
      )}  
      <h3>Total: ${total}</h3>  
    </div>  
  );  
};  
  
export default OrderSummary;
```

National University of Computer and Emerging Sciences
Islamabad Campus